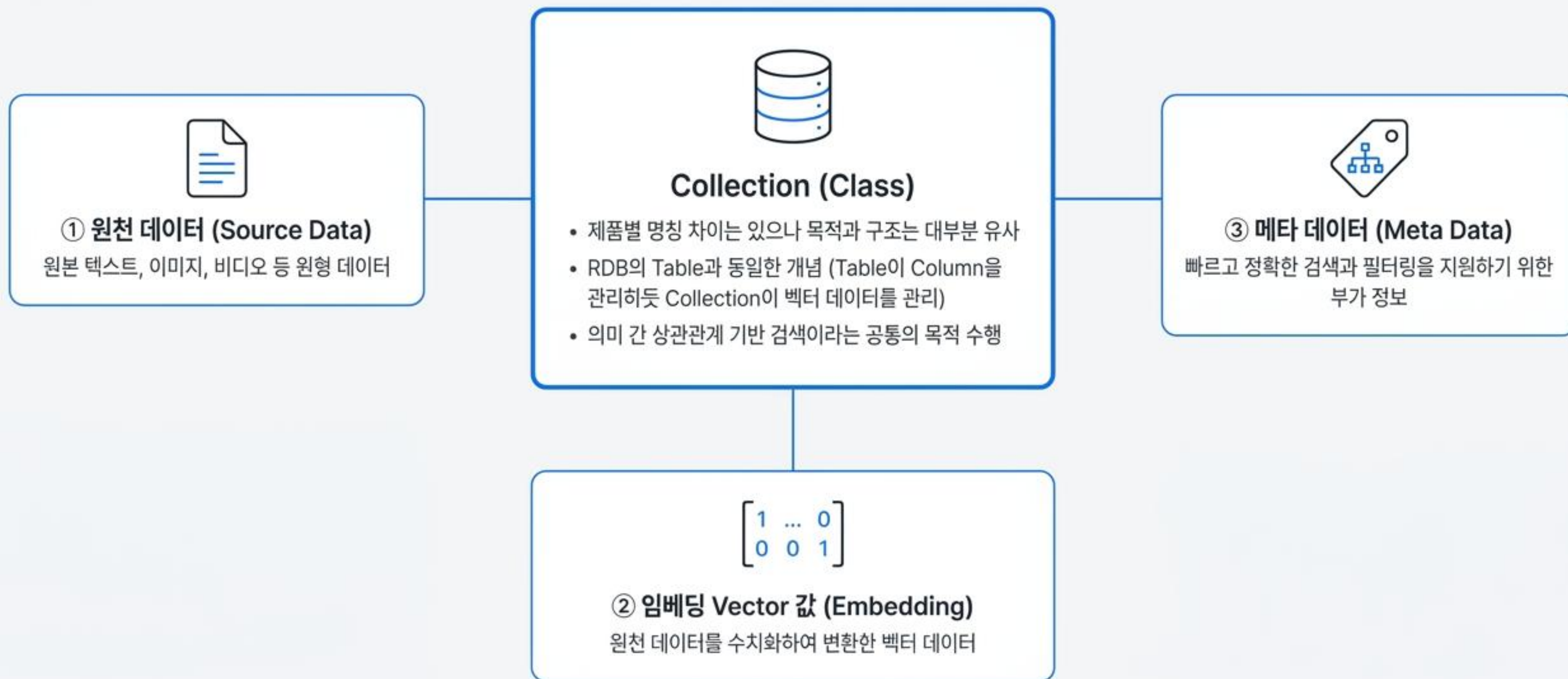




Vector DB 스키마와 설계

{ Vector • Graph 데이터베이스 }

Vector DB Schema 기본 구조



Collection Design 개요 및 단위

Database (논리적 그룹)



- Collection 집합의 최상위 논리적 그룹 단위
- 하나의 DB 내에 여러 개의 Collection 생성 및 운영 가능

Collection 관리 대상



- 임베딩 값 (Vector 데이터)
- 문서 (원천 데이터)
- 메타 데이터 (검색 지원용 속성)

Collection 도출 단위



- RDB의 Table 도출 단위와 매핑
- 객체지향 설계의 Class 도출 단위와 유사
- 유사한 데이터의 묶음이자 유사한 서비스 제공의 기준 단위

비정형 데이터의 유연성



- 정형 데이터와 마찬가지로 데이터 특성 및 유사도에 따라 유연한 설계 가능
- 비정형 데이터 역시 논리적 기준에 따라 분리 또는 통합 관리 가능

단일 Collection vs 멀티 Collection 설계

설계 방향성 요약

- 유사한 유형의 데이터를 하나로 묶을지, 여러 개로 나눌지에 대한 아키텍처 결정
- 관리의 편의성과 시스템 성능 사이의 트레이드오프(Trade-off) 고려 필수
- 데이터 규모에 따른 유연한 분산 배치 전략 필요



[Question]

유사한 유형의 데이터를 단일 Collection으로 설계할 것인가, 멀티 Collection으로 설계할 것인가?

[Answer]

일반적으로 단일 구조로 생각하기 쉬우나, 정확도 및 성능 향상을 위해 분리 설계를 채택하기도 함

- 소규모 데이터: 단일 Collection으로 통합 관리
- 대규모 데이터: 병렬 처리 및 성능 최적화를 위해 멀티 Collection으로 분산 배치

대용량 단일 Collection 관리의 장단점

단일 Collection 구조 분석



장점 (단순 구조)

- 모든 데이터가 한곳에 모여 관리가 직관적이고 단순함



단점 (성능 저하)

- 30TB 이상의 대규모 데이터 단일 관리 시 인덱싱, 검색, 업데이트 속도 급감

단점 (운영 리스크)

- 백업 및 복구 작업에 지나치게 긴 시간 소요

[결론] 테라급 대용량 데이터 분할

업무 유형 또는 일자 기준으로 멀티 Collection 분할이 유리

[기대 효과] 성능 및 운영 최적화

쿼리 성능 향상, 시스템 확장 용이성, 백업·복구 간편성 확보

[전제 조건] 인프라 및 자동화

분할로 인한 관리 복잡성 증가를 상쇄할 자동화 도구 및 적절한 인프라 확보 필수

이기종 데이터의 단일 Collection 통합 관리

[Question]

문자, 이미지, 동영상 등 전혀 다른 유형의 데이터를 단일 Collection으로 관리 가능한가?

[Answer]

가능함. 멀티모달(Multi-modal) 임베딩 기술을 통해 텍스트, 이미지, 오디오, 비디오를 동일한 벡터 공간에서 함께 이해하고 분석 및 관리 가능



장점

간소화된 검색 — 하나의 벡터로 문서·동영상의 복합 정보 표현

단점

복합 표현의 한계 — 서로 다른 정보 결합 과정에서 정보 손실 발생

벡터 크기 증가 — 저장공간·계산 비용 증가

활용 예시

이미지·텍스트 통합 검색 / 비디오 설명 생성 / 이미지·음성 입력 질의응답

Collection 설계 단위 결정 5대 기준

기준	내용	설계 시사점
① 데이터의 크기와 스케일링	대용량은 적절히 나누어 성능 유지·스케일링 용이화	4장의 규모 산정 결과가 그대로 입력
② 데이터의 접근 패턴	자주 접근하는 데이터끼리 묶어 설계	검색 로그·업무 시나리오 분석 선행
③ 데이터의 생명주기	생성·사용 시기별로 나누면 아카이브·삭제 용이	4장 데이터 최신성 관리와 직결
④ 데이터의 속성	비슷한 속성끼리 묶으면 검색·인덱싱 효율적	메타 필드 설계(5.4)와 함께 결정
⑤ 운영·관리의 용이성	Collection 수가 많으면 관리 복잡 → 적절한 단위	자동화 수준에 따라 감당 가능 개수 상이

Collection 설계 사례 — 레시피 데이터

데이터 특성 및 패턴 분석

[데이터 특성]

- 매일 등록되며 날짜별 등록 데이터량 다수 존재
- 요리 종류, 난이도, 작성자 등 다양한 메타 속성 보유

[데이터 접근 패턴]

- 최신 레시피의 검색 빈도 매우 높음 (오래된 레시피는 하락)
- 특정 종류(한식/양식) 및 특정 난이도 기반의 필터링 검색 빈번

[핵심 인사이트]

- 가장 적합한 설계 방법은 저장 규모와 실제 쓰임새 패턴에 따라 상이
- 분석 및 설계 단계에서 프로토타이핑을 통한 빠른 테스트 필수

레시피 설계 3가지 방법론

설계 방법	기준	장점	Collection 예시
날짜 기준	날짜별로 분할	최신 레시피 빠른 접근, 보관 용이	recipe_20240530
주제 기준	한식·중식·양식 등 종류별	특정 종류 검색이 빠르고 효율적	recipe_korean
혼합 기준	날짜+종류 (예: 매달 종류별)	과도한 세분화 없이 기간·종류 검색 효율	recipe_202405_korean

여러 Collection에 걸친 유사도 검색

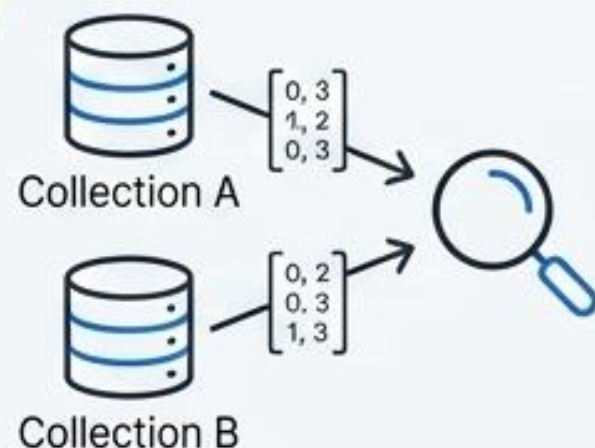
[Question]

서로 다른 Collection에 분산된 데이터 간의 유사도 검색이 가능한가?

[Answer]

일반적으로 가능함.

- (예시) 인덱스를 배열(Array) 형태로 지정하고 코사인 유사도 스크립트를 적용하여 교차 검색 수행



교차 검색을 위한 필수 전제 조건

벡터 공간의 일관성 (Consistent Vector Space) 유지 필수

- ✓ **동일한 차원 수**
768차원 모델 사용 시 검색 대상의 모든 벡터가 768차원이어야 함
- ✓ **동일한 임베딩 모델**
한쪽을 BERT로 벡터화했다면 비교 대상 항목도 반드시 동일 모델 적용
- ✓ **동일한 처리 방식**
토큰나이징 및 정규화(Normalization) 절차가 양쪽에 동일하게 적용되어야 함

⚠ **강조:** 서로 다른 모델의 벡터는 비교 불가 원칙

Vector Design — 벡터와 청크(Chunk)



[단계 1] 인식의 차이 (AI와 숫자의 언어)

- AI가 벡터를 사용하는 근본적 이유: 숫자가 컴퓨터의 모국어
- 사람은 언어와 이미지를 직관적으로 그대로 이해함
- 컴퓨터는 모든 데이터를 숫자의 배열(벡터)로 변환해야만 연산 및 이해 가능



[단계 2] 분할의 필요성 (문서 파편화)

- 방대한 크기의 문서 전체를 한꺼번에 단일 벡터로 변환하는 것은 불가능
- 컴퓨터가 처리할 수 있는 의미 단위, 혹은 모델의 제한 길이에 맞춰 여러 단위로 분할 필수



[단계 3] 청크(Chunk) 도출

- 청크(Chunk): 원천 데이터가 분할된 각각의 독립적 텍스트 단위
- 분할 완료된 이 '청크'를 최소 단위로 삼아 임베딩(벡터화) 수행

Sparse 벡터와 Dense 벡터 비교

1	0	
0	1	0
	0	1

Sparse 벡터 (희소 벡터)

- 구조: 데이터의 대부분의 값이 0으로 구성됨 (예: '김치' → [0, 1, 0, 0, 0])
- 특징: 특정 위치의 값(0이 아닌 값)만이 뚜렷한 의미를 지님
- 강점: 전통적인 키워드 기반 검색 및 정확한 단어 매칭에 매우 유리함



Dense 벡터 (밀집 벡터)

- 구조: 대부분 0이 아닌 연속적인 실수 값들로 뭉뚱하게 채워진 벡터
- 특징: 단어의 맥락과 숨겨진 의미적 관계를 공간상에 압축하여 표현
- 강점: 메모리 및 계산 효율이 높으며, 문맥 기반의 의미(Semantic) 기반 검색에 압도적으로 유리함



핵심 인사이트 (Hybrid Search)

- 컴퓨터는 검색 상황과 의도에 따라 Sparse와 Dense 방식을 선택적으로 사용
- 하이브리드 검색: Sparse(키워드 정확도)와 Dense(문맥적 의미)를 결합하여 동시에 사용하는 최신 검색 기법

Vector 구조 — Sparse와 Dense의 공존



기본 설계 구조

- 고유 ID + 원천 데이터 + 임베딩된 Vector 값으로 구성
- 최신 Vector DB 제품군 대부분은 하나의 Collection 내에 Sparse 벡터와 Dense 벡터의 동시 저장 및 관리를 지원함



① 다양한 인덱싱 전략 적용

Sparse 벡터와 Dense 벡터 각각의 특성에 가장 적합한 개별 인덱싱 알고리즘을 독립적으로 적용 및 최적화 가능



② 복합 검색 기능(Hybrid Query) 지원

상이한 벡터의 특성을 결합하여 다양한 검색 조건 동시 적용

(예시) 텍스트(키워드 중심) + 이미지(의미 중심) 동시 검색 수행



③ 무한한 확장성 보장

특정 데이터 타입이나 단일 저장 방식에 구애받지 않음

비즈니스 요구사항 변화에 맞춰 Collection을 유연하게 확장 및 고도화 가능

벡터 차원(Dimension)과 문장 길이의 독립성



입력 데이터 예시

짧은 문장:

'김치찌개는 맛있다.'

긴 문장:

'김치찌개는 신김치와
돼지고기를 함께 넣고 오래
끓일수록 맛있다...'



벡터 변환 결과

- 두 문장 모두 길이가 전혀 다름에도 불구하고 동일한 차원(예: 1,024차원)의 단일 벡터로 변환됨
- 긴 문장의 경우 더 많은 내부 토큰 연산을 거치며 전체 문맥을 복합적으로 고려할 뿐, 최종 출력되는 벡터의 그릇(차원크기)은 변하지 않음



핵심 인사이트

- 벡터의 차원은 원천 문장의 길이가 결정하는 것이 아님
- 벡터의 차원은 전적으로 사용된 임베딩 AI 모델의 아키텍처 스펙에 의해 고정됨

청킹 전략(Chunking Strategy) — Vector 단위 도출



청킹(Chunking)의 정의

- 에세이 → 단락 → 문장 단위와 같이 큰 텍스트를 더 작고 의미 있는 덩어리로 나누는 기법
- Vector Schema 설계의 최대 관건: '원천 데이터를 어떤 단위로 Vector화할 것인가?'



청킹 품질의 영향력

- 청킹의 품질이 검색 전 과정의 정확도를 좌우함
- 특히 사용자 질의(Query)와 시스템 청크 간의 유사성 매칭 성능에 직접적 영향



청킹 분할 방식 비교

- 휴리스틱 기반: 구두점(.), 줄바꿈, 문단의 끝 등 시각적·구조적 기호를 기준으로 분할
- 의미 기반: 텍스트 내러티브와 실제 의미 단위의 흐름으로 청크의 시작과 끝을 동적 결정



핵심 고려사항

- 긴 문장 임베딩: 디테일한 세부 정보와 의미 상실 우려
- 짧은 문장 임베딩: 전체 문맥 결여 및 정보 부족 현상 발생
- 결론: 적절한 청크 크기를 찾는 '문서 파편화 전략'이 검색 시스템 최적화의 핵심

청크 크기와 성능·정확도의 트레이드오프 관계

크기에 따른 성능 분석



① 응답 생성 시간 (Latency):

- 청크가 클수록 LLM에 한 번에 전달되는 정보(컨텍스트)량 증가
- 문맥은 풍부해지나 처리 토큰 증가로 인해 시스템 응답 속도 저하 위험 발생



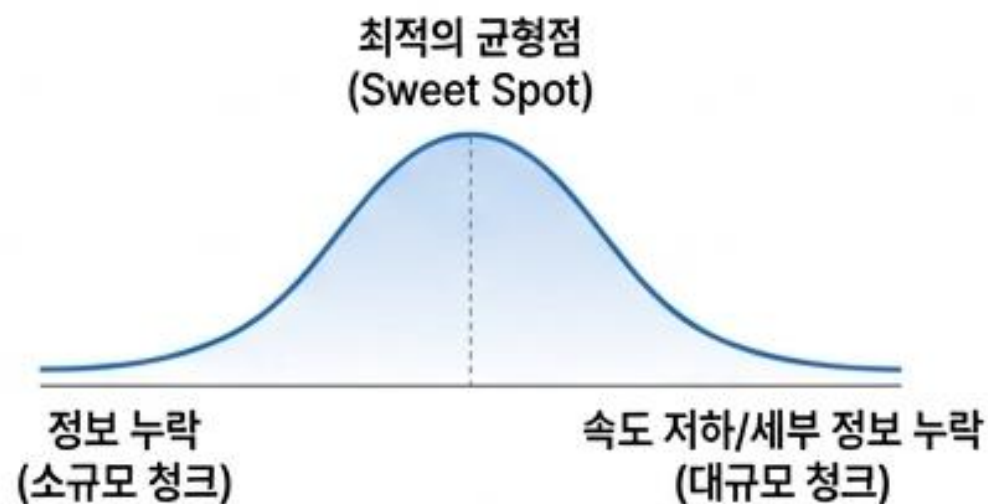
② 검색 정확성 (Accuracy & Recall):

- 작은 청크 (예: 128 토큰): 세분화된 매칭이 가능하나, 반환 개수(top_k)가 작을 경우 중요 맥락 누락 위험
- 큰 청크 (예: 512 토큰): 필요한 정보가 대부분 포함되어 전반적 정확성 향상. 단, 일정 한계 이상 커질 경우 정보가 과도하게 일반화·압축되어 디테일한 세부 정보 누락(정확도·재현율 하락) 초래



[목표] 최적의 균형점 탐색

정보 누락을 방지하면서도 응답 속도를 희생하지 않는 '골디락스(Goldilocks) 구간' 확보



[실행] 점진적 테스트 적용

- 소규모 데이터 샘플로 다양한 청크 크기를 먼저 테스트
- 성능과 정확도 지표 확인 후 전체 데이터로 확산 적용하는 것이 가장 현명한 실무적 접근

청킹 전략 수립 시 3대 고려사항

고려사항	판단 내용
① 색인 대상 콘텐츠의 성격은?	요리책 같은 긴 문서인가, 한 줄 후기 같은 짧은 콘텐츠인가 → 데이터 길이에 따라 적합한 임베딩 모델과 분할 전략이 확연히 달라짐
② 어떤 임베딩 모델을 쓰는가?	일반 범용 언어 모델은 개별 문장 단위 매칭에 강점이 있으나, 특정 특화 모델은 512 토큰 규모의 넓은 청크 컨텍스트에서 더 우수한 성능 발휘
③ 검색 결과를 어떻게 활용하는가?	단순 의미 검색인지, RAG 기반 질문 답변인지, 문서 요약인지 목적 정의 필수. 입력 토큰 제한이 있는 LLM에 공급할 시 전체 청크 수와 결합된 크기 제한을 엄격히 고려해야 함

데이터 맞춤형 청킹 전략 11가지

전략	설명	장점	단점
① Fixed Size	고정된 크기의 단어/문자 수로 일괄 분할	구현 단순, 균일 분포, 처리 시간 예측 가능	문맥적 비효율 발생 가능, 경계 영역의 의미 상실 우려
② Content Aware	텍스트의 내용 및 구조 분석 후 적절한 크기로 분할	데이터 특성 정밀 반영, 검색 효율 및 품질 향상	알고리즘 구현 복잡, 파싱을 위한 추가 계산 비용 발생
③ Overlapping	인접 청크 간의 텍스트 일부 중복(Overlap) 허용	경계선 정보 손실 방지, 검색 포괄성·정확성 증가	전체 저장공간 증가, 중복된 내용에 대한 관리 로직 필요
④ Recursive	재귀적으로 문서를 분할하여 계층적 트리 구조 생성	고도의 세분화 분석 가능, 계층적 문맥 검색 최적화	구현 난이도 매우 높음, 계층 간 데이터 일관성 관리 필요
⑤ Dynamic Size	문맥 및 내용의 밀도에 따라 청크 크기를 동적 조정	개별 데이터 맞춤 최적화, 스토리지 및 검색 효율 극대화	구현 복잡성 최상, 크기 편차로 인한 시스템 관리 어려움
⑥ Semantic	문단, 문장 등 의미론적 경계를 기준으로 분할	인간의 논리 구조와 일치하여 검색 정확도 대폭 향상	언어적 특성을 파악하는 복잡한 알고리즘 구현 필수
⑦ Hierarchical	상위 청크와 하위 청크의 부모-자식 계층 구조 매핑	넓은 문맥과 세부 정보 동시 확보, 빠르고 정확한 검색	DB 스키마 추가 설계 필요, 유지보수 공수 증가
⑧ Domain-Specific	법률, 의료 등 특정 도메인의 포맷 특성을 고려한 분할	특정 도메인 내에서 압도적인 검색 품질과 최적화 제공	타 비즈니스 도메인으로의 확장 및 재사용 적용 어려움
⑨ Summarization	원문을 LLM으로 요약한 결과를 청크로 구성	노이즈 제거로 검색 효율 향상, 대폭적인 저장공간 절약	요약 모델의 정확도에 전적으로 의존, 원본 디테일 정보 손실
⑩ Extract Candidate Q	문서가 대답할 수 있는 '예상 질문'을 추출해 청크로 구성	사용자 질문(Query)과의 매칭 확률 및 검색 정확성 극대화	질문 추출 프롬프트 구현 복잡, 사전 처리 알고리즘/비용 필요
⑪ Parent Child	요약(Parent) 검색 후 원문(Child)을 반환하는 계층 모델	계층 관계를 통한 맥락 보존, RAG 환경 검색 정확도 최고	설계 및 유지보수 복잡도 증가, 데이터 업데이트 시 재구성 필요

주요 청킹(Chunking) 전략의 분할 방식

전략	핵심 동작	사용 예시
① Fixed Size	- 길이·내용과 무관하게 크기를 일정 유지 - (예: 105개를 20 크기로 나누면 마지막은 5개)	- 한 줄이 독립적인 대량 주문 기록
② Content Aware	- 구두점·줄바꿈·NLP·마크다운 구조 기반 분할 - 문서 제목을 중간 청크에 추가해 손실 방지	- 긴 문서의 문단·의미 단위 분할 - 멀티미디어 장면 단위 분할
③ Overlapping	- 겹치는 부분을 두어 쿼리가 한 청크에 온전히 없어도 검색 가능성 확보	- 문단 간 연결 정보 보존이 필요한 문서 검색
④ Recursive	- 원하는 크기가 안 나오면 다른 구분 기호로 재분할 - (Fixed + Content Aware 혼합)	- 구조가 다양한 대형 문서
⑪ Parent Child	- 청크에 원문 미저장, 원문은 별도 외부 저장소 배치 - 검색된 청크의 포인터로 원문 획득	- 책 → 챕터(부모)·구절(자식) - 웹사이트 → 섹션·하위 페이지

Semantic Chunking 상세 동작 구조

Card 1

단계 1: 텍스트 분할

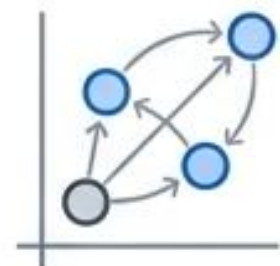
- 구조적 요소가 아닌 '의미'를 기준으로 텍스트 그룹화
- 텍스트를 문장 단위 등 기초 단위로 청크화



Card 2

단계 2: 벡터 임베딩 및 유사도 연산

- 각 청크를 벡터 임베딩(Vector Embedding)으로 변환
- 생성된 청크 간의 코사인 유사성(Cosine Similarity) 계산



Card 3

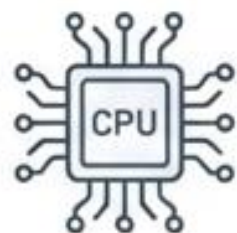
단계 3: 조건부 통합 및 분리

- 설정된 임계값(예: 0.8)을 기준으로 통합 또는 분리 판별
- 의미가 유사하면 하나의 청크로 통합, 다르면 개별 청크로 분리 유지



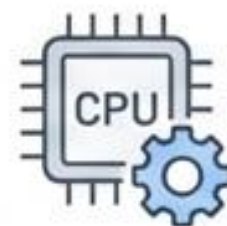
주요 임베딩 모델별 입력 길이 제한

BERT 모델



- 최대 한도: 최대 512 토큰 제한
- 기준: 단어 수가 아닌 '토큰 수' 기준
- 특징: 하나의 단어가 여러 토큰으로 분할 가능
(understanding → under/stand/ing)

RoBERTa 모델



- 최대 한도: BERT와 유사하게 최대 512 토큰 제한
- 특징: 최적화된 사전학습 기법이 적용된 모델 구조 공유

GPT 계열 모델



- 용량: 수천 ~ 수십만 토큰 처리 가능
- 한계: 입력 길이가 길어질수록 처리 비용 및 지연 시간(Latency) 증가

핵심 설계 원칙



- 청크 크기 정의: "몇 글자"가 아니라 "몇 토큰"으로 정의
- 필수 확인: 설계 전 각 임베딩 모델의 하드웨어 제한 사항 최우선 확인

메타 데이터(Meta Data) 디자인의 3대 핵심 역할



① 검색 효율성 향상

- 문제: 문서 증가 시 벡터 검색 회수율 급감
- 해결: 생성 날짜, 소스, 카테고리 등 메타 필터링으로 검색 범위 축소 및 정확도 지원
- 효과: 금융 등 유사 콘텐츠가 많은 환경에서 잘못된 문단(오탐) 반환 문제 완화



② 보안 및 접근 제어

- 목적: 소유자, 권한 레벨, 민감도 수준 저장을 통한 시스템 보안 강화
- 적용: 조회 시 사용자의 권한 메타데이터와 대조 수행
- 예시: 레시피 문서에 "회원등급: 실버 이상" 메타를 설정하여 조회 권한 제한



③ 관리 및 부가정보

- 목적: 데이터 출처, 업데이트 기록, 버전 정보 저장
- 효과: 데이터 생애 주기 추적, 벡터 DB 내 정합성 유지 및 체계적 관리 가능

메타 데이터의 주요 활용 분야

검색 및 필터링



- 특정 카테고리, 날짜 범위, 태그를 기반으로 타겟 벡터를 정밀 검색 및 필터링

데이터 시각화



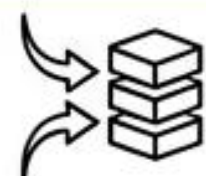
- 메타 데이터를 활용하여 추상적인 벡터 데이터를 시각적으로 표현 및 매핑 분석

추천 시스템



- 메타 데이터 속성을 결합하여 사용자 의도에 부합하는 높은 관련성의 추천 결과 제공

데이터 통합



- 이기종 환경 등 다양한 소스에서 수집된 데이터를 단일 기준으로 통합 및 관리

- 핵심 요약: 메타 데이터는 벡터의 '컨텍스트(Context)'를 이해하고 실무적으로 활용하는 데 필수적인 중추 역할을 수행함

메타 데이터 표준 미준수 시 리스크

표준 미준수 파급 효과

- 벡터 DB 내 데이터 일관성 심각한 훼손
- 데이터의 체계적 관리 및 검색 곤란
- 하위 애플리케이션의 분석 오류 및 시스템 장애 초래
- 핵심 결론: "일관성 부족은 필요한 데이터를 찾을 수 없게 만든다"

⚠ 사례 1: 필드명 비표준화

- 현상: itemCategory / category / product_category 혼재
- 결과: "Electronics" 검색 시 쿼리 1개가 아닌 3개가 동시 필요 (비표준 증가 시 부하 가중)

⚠ 사례 2: 데이터 값 비표준화

- 현상: "Electronics"를 "Elec" 등 임의의 약어로 저장
- 결과: 정확한 키워드 검색 시 결과가 '빈 배열(Empty Array)'로 반환됨

메타 표준 설계 가이드라인

Context Panel

설계 확정 핵심 원칙



- 원칙: 메타 표준은 반드시 '설계 초기 단계'에서만 확정 가능
- 사후 변경 리스크: 운영 중 필드명이나 값 변경하려면 이미 적재된 모든 문서의 재수정 및 재색인 불가피

1. 필드명 명명 규칙 확정



- snake_case 또는 camelCase 중 단일 규칙으로 전사적 통일

2. 값의 도메인(Domain) 통제



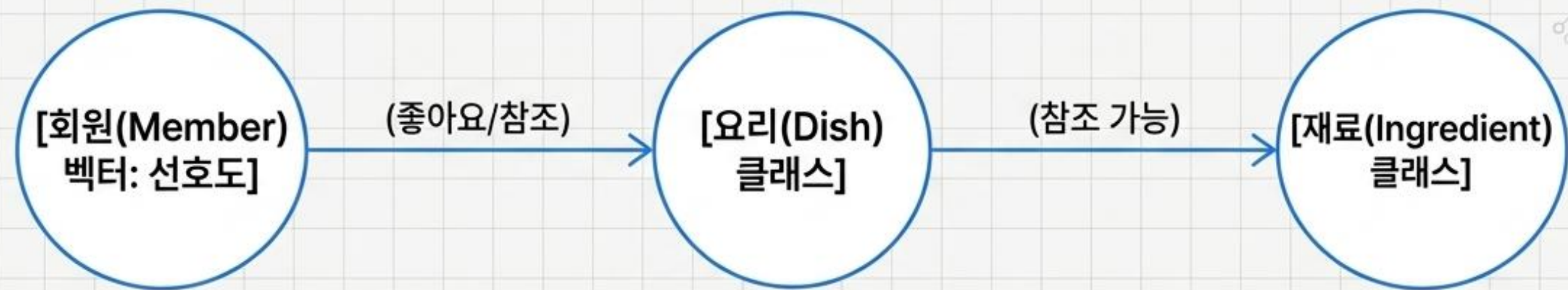
- 사전 정의된 코드값 목록 사용
- 날짜 표기 형식(Format) 및 대소문자 입력 규칙 엄격 통제

3. 필수/선택 필드 명확화



- 필수(Mandatory) 지정 권장 속성: 권한, 출처, 유효기간 등

벡터 데이터베이스의 객체 관계(Relationship) 설계



관계형 벡터 설계 원리

- 요리와 회원의 선호도를 각각 독립된 벡터로 저장
- 유사도 검색(Similarity Search)을 통해 회원의 선호도 벡터와 가장 유사한 요리 추천

참조 벡터(ref2vec) 활용

- 일부 고도화된 Vector DB는 클래스 간 관계 자체를 벡터로 표현 지원
- 회원이 "좋아요"한 요리의 참조 값을 체계적으로 관리하여 다대다 관계 생성
- 요리 클래스는 하위 재료 클래스를 다시 참조하는 계층적 관계망 구축 가능

중심 벡터(Centroid) 계산 원리 및 요리 추천 스키마



중심 벡터(Centroid) 계산 원리

- 장점: 참조 객체가 다수여도 벡터를 평균화하여 하나로 간략히 표현 (검색 효율성 극대화)

단계	동작
① 하위 객체 벡터 생성	- 각 조리단계(Step)가 자체 벡터 보유, 단계별 고유 의미 표현
② 상위 객체 벡터 계산	- 레시피(Recipe)가 참조하는 조리단계들의 벡터 '평균' 산출
③ 저장 및 ④ 검색	- 계산된 벡터를 상위 객체와 연결하여 저장 후, 레시피 검색 시 대표 벡터로 우선 사용



요리 추천 스키마 및 동작 방식 예시

클래스	주요 속성(스키마)	동작 방식
요리 (Dishes)	dish_id, name, category, description, cook_time, embedding	- 설명·종류·시간 기반 NLP 모델로 임베딩 벡터 생성
회원 (Members)	member_id, name, age, gender, preferences(선호 평균 벡터)	- 좋아하는 요리 임베딩을 평균·가중 평균 산출
선호 (Relationship)	member_id, dish_id, rating, timestamp	- 별점 갱신 시 선호 벡터 재산출 → 코사인/유클리드 거리로 추천

2026 실무 가이드 ①: 청킹 구성의 중요성과 기본값

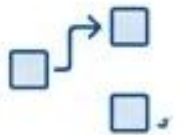
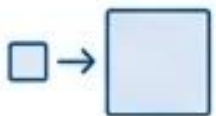
임베딩과 청킹의 상관관계

- 현실: 모델 선택에만 집중하고, 청킹은 기본값(Default)으로 방치하는 경우 다수
- 최신 연구: 청킹 구성의 영향력이 임베딩 모델 선택만큼(혹은 그 이상) 검색 성능에 직결됨
- 결과: 동일한 코퍼스(Corpus) 내에서도 최선/최악의 청킹 전략 간 재현율 격차 상당함
- 주의: 제시된 기본값은 시작점이며, 최종 근거는 자체 평가셋 측정 결과가 되어야 함

항목	2026년 실무 기본값	근거
청크 크기	256~512 토큰에서 시작	- 작으면 문맥 상실 - 크면 관련성 희석
오버랩	청크 크기의 10~25% (예: 512 토큰에 128 토큰)	- 경계 구역 정보 손실 방지
크기 단위	문자 수가 아닌 '토큰 수' 기준	- 모델 제한 및 비용 기준 부합
상한선 주의	2,500 토큰 부근에서 응답 품질 급락 구간 관측	- 큰 청크가 항상 좋은 것은 아님

2026 실무 가이드 ②: 프로덕션 표준 '계층적 청킹'

청크 크기의 딜레마와 해결 원리

-  - 작은 청크의 한계: 검색은 정밀하지만 문맥이 불완전함
-  - 큰 청크의 한계: 문맥은 완전하지만 검색 지점이 부정확함
-  - 해결 원리: 찾을 때는 정밀한 '작은 단위'로, LLM이 이해할 때는 그 청크가 속한 '큰 문맥'으로 전달

구현: Parent-Child 전략

-  - 검색(Search): 잘게 쪼개진 자식(Child) 단위로 고정밀 검색 수행
-  - 획득(Retrieve): 검색된 자식의 '부모 ID'를 활용하여 전체 원문 컨텍스트 획득

데이터베이스 설계 함의

-  - 메타 필드: parent_id 계열의 관계형 메타 필드 설계 필수
-  - 스토리지: 무거운 원문은 벡터 DB 내부가 아닌 '외부 문서 저장소'에 분리 배치

2026 실무 가이드 ③: 청크 경계의 문맥 손실 해결

- 문제 상황: "그것은 10분 더 끓이면 된다" → 단일 청크만으로는 주어(무엇) 판별 불가
- 파급력: 대명사·상호 참조가 많은 사용 설명서, 후기, 계약서 등에서 흔한 검색 실패 유발

기법	동작 방식	장점	비용 및 한계
문맥 보강 청킹 (Contextual Retrieval)	- 각 청크 앞에 LLM이 생성한 문서 수준 요약 문맥을 덧붙여 임베딩	- 기존 모든 기본 전략에 유연하게 적용 가능 - 성능 향상 폭 큼	- 청크마다 LLM 호출 비용 발생 - (프롬프트 캐싱으로 절감 필요)
늦은 청킹 (Late Chunking)	- 문서 전체를 통째로 토큰 단위 임베딩한 뒤, 청크 경계를 적용하여 평균 풀링	- 비싼 LLM 호출 비용 없이 임베딩 모델 자체만으로 구현 가능	- 긴 문맥 윈도우를 지원하는 모델 필수 - (모델 의존성 매우 높음)

2026 실무 가이드 ④: 문서 유형별 청킹 라우팅 전략

문서 유형	권장 청킹 방식	설계 이유
레시피·매뉴얼 (조·항 구조)	- 구조 인식 기반 + 계층적 청킹	- 각 조리단계 및 조항 자체가 독립된 '의미 단위'이자 '인용 단위'임
보고서 PDF (표·그림 다수)	- 레이아웃 인식 처리 - 표(Table)는 별도 청크 분리	- 구조화된 표를 문장 사이에 섞어 임베딩 시 양쪽 데이터 모두 심각하게 훼손됨
게시판 Q&A 데이터	- 질문(Q)과 답변(A) 쌍을 결합하여 '하나의 청크'로 구성	- 임의 분리 시 쿼리 수행 단계에서 답변만 검색되어 본래 맥락 상실 위험
프레젠테이션 (PPT 등)	- 개별 슬라이드 단위 청킹 - 제목 경로(Path) 부착	- 작성자가 이미 슬라이드 1장을 독립된 '논리적 완결 단위'로 설계했기 때문
소스코드·설정 파일 (IT)	- 함수(Function) 및 코드 블록 경계 인식 분할기 사용	- 프로그래밍 문법 단위를 강제로 자르면 로직의 의미 붕괴

2026 실무 가이드 ⑤: 메타 스키마 분류별 필수 필드

분류 카테고리	설계 필드 예시	실무 적용 용도
식별 및 계보 추적	doc_id, chunk_id, parent_id, source_path, page_no	- 출처 링크 제공, 계층적 검색, 부분 삭제/갱신
보안 및 거버넌스	allowed_roles, dept, sensitivity, valid_until	- 권한별 결과 필터링, 만료된 문서 자동 제외
데이터 신선도 유지	created_at, updated_at, doc_version	- 최신 문서 우선 정렬, 시스템 증분 동기화 기준
파이프라인 처리 이력	parser_version, embedding_model, chunk_strategy	- 향후 재임베딩 대상 판별, 실험 간 성능 비교
논리적 분류체계	doc_type, category, title_path	- 메타 필터링 수행, 검색 결과 그룹핑(Grouping)

- 아키텍처 핵심: embedding_model과 chunk_strategy를 메타 데이터로 기록 필수.
향후 AI 모델이나 분할 전략 교체 시, 전체 재처리가 아닌 '재처리 필요 대상'만 즉각 판별하여 마이그레이션 가능

벡터 DB 설계 시점에 확정해야 하는 5대 항목

- 주의: 아래 항목들은 사후 변경 시 시스템 재색인 또는 전체 데이터 재적재(비용 급증)를 유발함.
프로젝트 설계 종료 전 반드시 명문화 및 문서화 필수.



1. Collection 아키텍처

- Collection 분리 기준 및 전사적 명명 규칙(Naming Convention) 확정



2. 임베딩 및 인덱스 스펙

- 적용 모델, 차원 수, 유사도 척도(Cosine/Dot/L2), 인덱스 종류(HNSW/IVF) 지정



3. 청킹(Chunking) 정책

- 적용 전략, 기본 크기, 오버랩 비율 확정 (반드시 '토큰' 기준 명시)



4. 메타 데이터 스키마

- 전체 필드 목록, 값의 도메인(규칙), 필수/선택 속성 구분 명문화



5. 데이터 스토리지 전략

- 원문(Raw Text) 저장 위치 결정 (벡터 DB 내부 저장 vs 외부 전용 문서 저장소 배치)